

Desafio Técnico

Sistema de Gestão de Ordens de Venda (OVGS)

Objetivo

Este desafio tem como objetivo avaliar a capacidade de projetar, implementar e documentar uma solução para gestão do ciclo de vida de Ordens de Venda (OVs), considerando aspectos de modelagem de domínio, aplicação de regras de negócio, arquitetura de software, qualidade de código e tomada de decisão técnica.

Mais do que a implementação funcional dos requisitos, buscamos compreender como o candidato estrutura soluções para problemas de negócio reais, realiza escolhas arquiteturais e equilibra requisitos de manutenibilidade, escalabilidade e evolução do sistema.

Contexto de Negócio

Atualmente, a gestão de Ordens de Venda é realizada através de múltiplos sistemas e controles operacionais independentes.

Esse cenário gera desafios relacionados a:

- Rastreabilidade das operações;
- Visibilidade do fluxo logístico;
- Controle dos agendamentos de entrega;
- Consistência das informações;
- Governança sobre alterações realizadas ao longo do processo.

Como iniciativa de modernização, a empresa pretende centralizar essas operações em uma plataforma única capaz de gerenciar o ciclo completo das Ordens de Venda.

Escopo do Desafio

A solução deverá contemplar a gestão completa das Ordens de Venda, incluindo:

- Cadastro de clientes;
 - Cadastro de tipos de transporte;
 - Cadastro de itens;
 - Criação e acompanhamento de Ordens de Venda;
 - Agendamento de entregas;
 - Auditoria das principais alterações realizadas no sistema.
-

Perfis Avaliados

Desenvolvedor Back-end

Implementar:

- API REST;
- Modelagem de domínio;
- Persistência de dados;
- Regras de negócio;
- Auditoria;
- Testes automatizados;
- Documentação técnica.

Não é necessária a implementação de interface gráfica.

Desenvolvedor Front-end

Implementar a interface da solução contemplando:

- Gestão de Ordens de Venda;
- Monitoramento operacional;
- Central de Agendamento;
- Cadastros básicos;
- Integração com APIs;
- Tratamento de estados;
- Validações de entrada.

A utilização de APIs simuladas (mockadas) é permitida.

Desenvolvedor Full Stack

Implementar os requisitos previstos para Front-end e Back-end.

Regras de Negócio

Cliente

Cada cliente poderá possuir uma lista de tipos de transporte autorizados.

Uma Ordem de Venda somente poderá ser criada caso o tipo de transporte informado esteja previamente autorizado para o cliente selecionado.

Tipo de Transporte

O sistema deverá permitir o cadastro de diferentes modalidades de transporte.

Exemplos:

- Caminhão
- Carreta
- Bi-truck

A inclusão de novos tipos de transporte deverá ocorrer sem necessidade de alteração das regras de negócio existentes.

Ordem de Venda

Uma Ordem de Venda deverá:

- Estar vinculada a um único cliente;
 - Possuir exatamente um tipo de transporte;
 - Conter ao menos um item;
 - Possuir um status válido dentro do fluxo operacional.
-

Itens

Os itens utilizados nas Ordens de Venda deverão estar previamente cadastrados e possuir um identificador único (por exemplo, SKU).

Ao criar uma Ordem de Venda, deverá ser possível associar um ou mais itens previamente cadastrados.

A forma de modelagem dos itens, seus atributos, relacionamentos e eventuais mocks necessários para a implementação ficam a critério do candidato. Entretanto, a solução deve considerar como premissa que os itens existem previamente no sistema e podem ser vinculados às Ordens de Venda.

Fluxo Operacional da Ordem de Venda

O sistema deverá suportar os seguintes estados:

- CRIADA
- PLANEJADA
- AGENDADA
- EM_TRANSPORTE
- ENTREGUE

Somente transições válidas deverão ser permitidas.

Fluxo esperado:

CRIADA → PLANEJADA → AGENDADA → EM_TRANSPORTE → ENTREGUE

Tentativas de transição fora da sequência definida deverão ser rejeitadas e devidamente tratadas.

Funcionalidades Esperadas

Gestão de Ordens de Venda

Permitir:

- Criar Ordem de Venda;
 - Consultar Ordens de Venda;
 - Consultar detalhes de uma Ordem de Venda;
 - Atualizar status da Ordem de Venda.
-

Monitoramento Operacional

Disponibilizar consultas com filtros por:

- Status;
 - Cliente;
 - Tipo de transporte;
 - Data.
-

Central de Agendamento

Permitir:

- Definição da data de entrega;
- Definição de janela de atendimento;
- Confirmação do agendamento;
- Reagendamento.

As regras de disponibilidade poderão ser simplificadas ou simuladas.

Cadastros

Clientes

- Criar;
- Editar;
- Consultar.

Tipos de Transporte

- Criar;
- Editar;
- Consultar.

Itens

- Criar;
 - Consultar.
-

Auditoria

A solução deverá registrar eventos relevantes para fins de rastreabilidade.

Eventos mínimos esperados:

- Criação de Ordem de Venda;
- Alteração de status;
- Alteração de agendamento;
- Alteração de transporte.

Cada evento deverá registrar:

- Data e hora;
 - Tipo de ação;
 - Entidade afetada;
 - Estado anterior (quando aplicável);
 - Estado posterior (quando aplicável).
-

Requisitos Técnicos

Tecnologias Obrigatórias

- Node.js
 - TypeScript
 - NestJS
 - Banco de dados relacional
 - Prisma, TypeORM ou Sequelize
 - Docker Compose
-

Estrutura Esperada

A organização da solução deverá evidenciar separação adequada de responsabilidades, incluindo, quando aplicável:

- Controllers;
- Services;
- Repositories;
- DTOs;
- Camadas de validação;
- Tratamento de exceções;
- Estratégias de persistência.

A adoção de padrões arquiteturais adicionais fica a critério do candidato e deverá ser devidamente justificada.

Testes

Implementar, no mínimo:

- 2 testes unitários;
- 1 teste de integração.

Os testes deverão contemplar cenários relevantes das regras de negócio.

Será considerado diferencial apresentar estratégia mais abrangente de cobertura.

Documentação

O repositório deverá conter um README contendo:

- Instruções de execução;
 - Tecnologias utilizadas;
 - Decisões arquiteturais;
 - Estratégia de modelagem do domínio;
 - Estratégia de persistência;
 - Considerações sobre escalabilidade;
 - Considerações sobre performance;
 - Trade-offs assumidos.
-

Diferenciais

Os seguintes itens serão considerados diferenciais, desde que façam sentido para a solução proposta:

- OpenAPI / Swagger;
 - Clean Architecture;
 - Event-Driven Architecture;
 - Logs estruturados;
 - Observabilidade;
 - Métricas e monitoramento;
 - Estratégias de cache;
 - Otimização de consultas;
 - Testes adicionais;
 - Pipeline de CI/CD;
 - Estratégias de segurança e autorização.
-

O Que Esperamos de um Desenvolvedor Sênior

Durante a avaliação serão observados aspectos que vão além da implementação funcional:

- Capacidade de modelar corretamente o domínio;
- Clareza na separação de responsabilidades;
- Qualidade das decisões arquiteturais;
- Capacidade de justificar trade-offs;
- Preocupação com escalabilidade e manutenção;
- Qualidade da documentação;
- Cobertura e relevância dos testes;
- Legibilidade e consistência do código.

Não é esperado que todos os diferenciais sejam implementados. A consistência técnica da solução será considerada mais relevante do que a quantidade de tecnologias utilizadas.

Entregáveis

Disponibilizar um repositório Git contendo:

- Código-fonte;
- Docker Compose;
- Scripts necessários para execução;
- README;
- Documentação complementar, quando aplicável.